

SHOPEZ

E-Commerce Application

Full Stack Development with MERN

Team Members:

Bhanusri Manukonda(Team Lead)
Rajasekhar Rowlo
Jayanti Das Suhani
Pithala Vikas

1. Introduction

Team Members

Team Member	Role
Bhanusri Manukonda	Frontend Developer — React.js
Rajasekhar Rowlo	Backend Developer — Node.js / Express.js
Jayanti Das Suhani	Database & API Integration — MongoDB
Pithala Vikas	Authentication & Testing

SHOPEZ is a full-stack E-commerce web application developed using the MERN Stack (MongoDB, Express.js, React.js, and Node.js). The project aims to provide customers with a seamless online shopping experience by offering a responsive user interface, secure authentication, efficient product browsing, shopping cart management, and order processing functionalities.

The application allows users to create accounts, securely log in, browse products by category, search for products, add items to the shopping cart, place orders, and manage their profiles. Administrators are provided with a dedicated dashboard to manage products, categories, users, inventory, and customer orders.

The project follows a client-server architecture where React.js is used to develop the frontend, Node.js and Express.js are used to build RESTful APIs, and MongoDB serves as the database for storing application data. Git and GitHub are used for version control and collaborative development throughout the project lifecycle.

This project helped the team gain practical knowledge in full-stack web development, API integration, database management, authentication using JSON Web Tokens (JWT), responsive UI design, and collaborative software development practices.

2. Project Overview

Purpose

The primary purpose of the SHOPEZ E-commerce Application is to develop a modern, secure, and user-friendly online shopping platform where customers can easily browse products, add them to a shopping cart, and complete purchases efficiently. The application is designed to simplify the shopping process while providing administrators with tools to manage products, users, and orders effectively.

The project also serves as a practical implementation of MERN Stack technologies, enabling the development team to gain hands-on experience in frontend development, backend development, REST API creation, database management, authentication, and deployment.

Project Goals

- Develop a responsive and interactive web application.
- Provide secure user registration and authentication.
- Enable customers to search and browse products efficiently.
- Implement shopping cart and order management functionalities.
- Build scalable RESTful APIs using Express.js and Node.js.
- Store and manage application data using MongoDB.
- Create an admin dashboard for managing products, users, and customer orders.
- Ensure smooth communication between frontend and backend through APIs.

Key Features

User Module

- User Registration
- Secure Login using JWT Authentication
- User Profile Management
- Password Protection

Product Module

- Product Listing
- Product Categories
- Product Details
- Product Search and Filtering

Shopping Cart

- Add Products to Cart
- Update Product Quantity
- Remove Products from Cart
- View Cart Summary

Order Management

- Place Orders
- Order Confirmation
- Order History
- Order Status Tracking

Admin Module

- Product Management

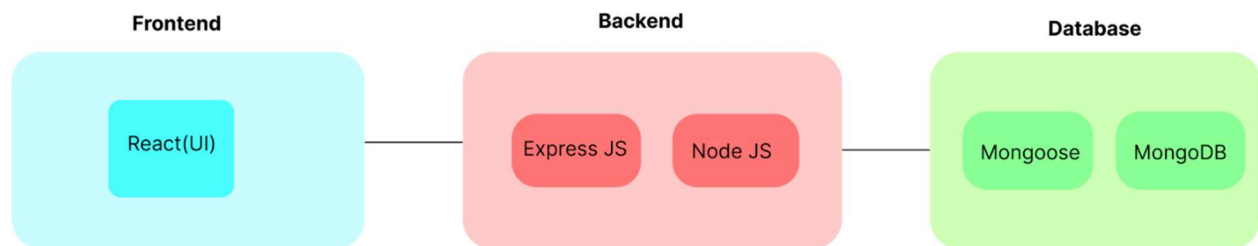
- User Management
- Order Management
- Dashboard Analytics

Additional Features

- Responsive Design
- RESTful API Integration
- MongoDB Database
- Error Handling and Validation
- Secure Authentication and Authorization

3. Architecture

The SHOPEZ E-commerce Application follows a three-tier architecture consisting of the Presentation Layer (Frontend), Application Layer (Backend), and Database Layer. This architecture separates concerns, improves scalability, and simplifies maintenance.



Frontend Architecture (React.js)

The frontend of the application is developed using React.js, a component-based JavaScript library that enables the creation of dynamic and responsive user interfaces. The application is divided into reusable components such as Navbar, Product Cards, Cart, Login, Registration, Checkout, and Admin Dashboard.

React Router is used for client-side routing, allowing users to navigate between different pages without refreshing the browser. Axios (or Fetch API) is used to communicate with backend REST APIs. State management is handled using React Hooks and Context API (or Redux, if implemented) to efficiently manage application data across components.

Frontend Responsibilities

- User Interface Rendering
- User Authentication
- Product Display
- Shopping Cart Management
- Order Placement
- API Communication
- Form Validation

- Responsive Design

Backend Architecture (Node.js & Express.js)

The backend is developed using Node.js and Express.js, following RESTful architecture principles. Express.js provides routing, middleware, request handling, and API management.

The backend processes client requests, validates input data, authenticates users using JSON Web Tokens (JWT), performs business logic, interacts with the MongoDB database, and returns JSON responses to the frontend.

Backend Responsibilities

- User Authentication and Authorization
- Product Management APIs
- Cart Management
- Order Processing
- Database Operations
- Error Handling
- API Security
- Request Validation

Database Architecture (MongoDB)

MongoDB is used as the primary NoSQL database to store application data. Data is organized into collections containing documents in JSON format, providing flexibility and scalability.

Major Collections

- Users
- Products
- Categories
- Shopping Cart
- Orders
- Admin

Each collection is connected logically using document references where required. Mongoose is used as the Object Data Modeling (ODM) library to define schemas, validate data, and interact with MongoDB efficiently.

Database Responsibilities

- Store User Information
- Store Product Details
- Maintain Shopping Cart Data
- Store Customer Orders
- Manage Product Categories

- Support CRUD Operations
- Ensure Data Integrity

4. Setup Instructions

Prerequisites

Before setting up the SHOPEZ E-commerce Application, ensure that the following software and tools are installed on your system:

- Operating System: Windows 10/11, macOS, or Linux
- Node.js: Version 16 or above
- npm (Node Package Manager): Version 8 or above
- MongoDB: Community Edition or MongoDB Atlas
- Git: For version control
- Visual Studio Code: Recommended code editor
- Postman: For testing REST APIs
- Google Chrome or Mozilla Firefox: Latest version for running the application

Installation

Step 1: Clone the Repository

```
git clone <repository-url> cd shopez
```

Step 2: Install Frontend Dependencies

Navigate to the client folder and install the required packages.

```
cd client npm install
```

Step 3: Install Backend Dependencies

Open another terminal, navigate to the server folder, and install the backend packages.

```
cd server npm install
```

Step 4: Configure Environment Variables

Create a .env file inside the server directory and configure the following variables:

```
PORT=5000 MONGO_URI=your_mongodb_connection_string  
JWT_SECRET=your_secret_key
```

Replace the placeholder values with your MongoDB connection string and secure JWT secret key.

Step 5: Verify MongoDB Connection

Ensure that MongoDB is running locally or that your MongoDB Atlas cluster is accessible before starting the backend server.

Step 6: Install Additional Packages (if required)

If any dependency is missing, install it using npm.

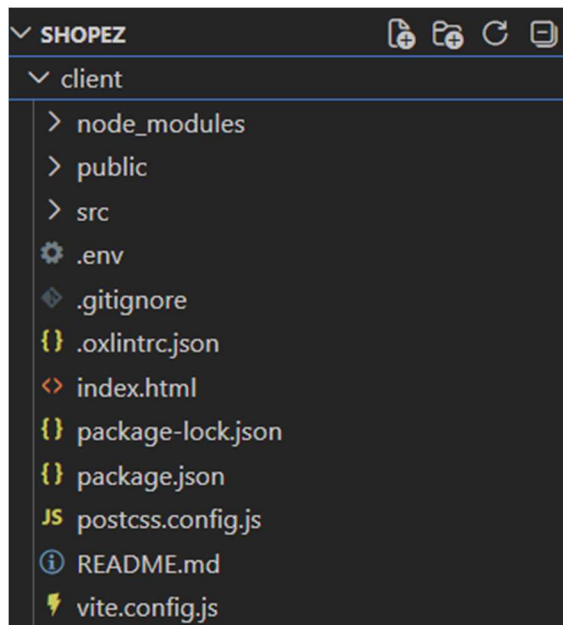
```
npm install axios npm install jsonwebtoken npm install bcryptjs npm  
install mongoose
```

After completing these steps, the application is ready to run locally.

5. Folder Structure

The SHOPEZ project is divided into two major directories: client and server. This separation simplifies development and maintenance by organizing frontend and backend code independently.

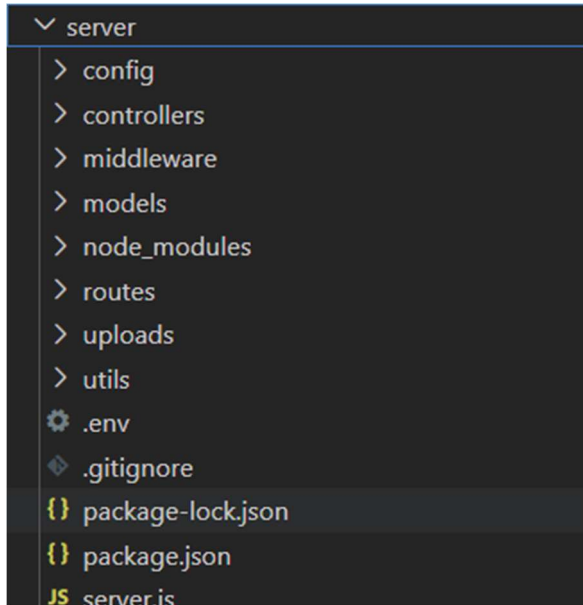
Client (React Frontend)



Description

- public/ – Static files served directly by the browser.
- src/ – Main source code of the React application.
- components/ – Reusable UI components such as Navbar, Footer, Product Card, and Cart.
- pages/ – Individual application pages like Home, Login, Products, Cart, Checkout, and Admin Dashboard.
- services/ – API communication using Axios or Fetch.
- context/ – Global state management using React Context API.
- assets/ – Images, icons, and other static resources.
- styles/ – CSS files and styling resources.

Server (Node.js Backend)



Description

- config/ – Database connection and application configuration.
- controllers/ – Business logic for handling API requests.
- middleware/ – Authentication, authorization, and error-handling middleware.
- models/ – MongoDB schemas defined using Mongoose.
- routes/ – REST API endpoint definitions.
- utils/ – Helper functions and reusable utilities.
- server.js – Entry point of the backend application.

6. Running the Application

After successfully installing all dependencies and configuring the environment variables, the application can be started locally.

Start the Backend Server

Open a terminal and navigate to the server directory.

```
cd server npm start
```

Alternatively, if nodemon is installed:

```
npm run dev
```

The backend server will start on:

```
http://localhost:5000
```

Start the Frontend Application

Open another terminal and navigate to the client directory.

```
cd client npm start
```

The React development server will launch automatically and open the application in the default web browser.

Frontend URL:

```
http://localhost:3000
```

Application Workflow

1. Start the MongoDB server or connect to MongoDB Atlas.
2. Run the backend server.
3. Start the React frontend.
4. Open the application in a web browser.
5. Register a new user or log in with existing credentials.
6. Browse products, add items to the cart, and place orders.
7. Administrators can log in to manage products, users, and customer orders through the admin dashboard.

Verification

To verify that the application is running correctly:

- Ensure the frontend loads without errors.
- Confirm that the backend connects successfully to MongoDB.
- Test user registration and login.
- Verify product listing and search functionality.
- Add products to the cart and place a sample order.
- Test API endpoints using Postman to ensure proper backend responses.

7. API Documentation

The backend of the SHOPEZ E-commerce Application exposes a set of RESTful APIs that enable communication between the React frontend and the MongoDB database. These APIs handle user authentication, product management, shopping cart operations, and order processing.

User APIs

Method	Endpoint	Description
POST	/api/users/register	Register a new user

Method	Endpoint	Description
POST	/api/users/login	Authenticate user and generate JWT token
GET	/api/users/profile	Retrieve logged-in user profile
PUT	/api/users/profile	Update user profile information

Example Request (User Login)

```
{  "email": "user@example.com",  "password": "password123" }
```

Example Response

```
{  "success": true,  "token": "JWT_TOKEN",  "message": "Login Successful" }
```

Product APIs

Method	Endpoint	Description
GET	/api/products	Retrieve all products
GET	/api/products/:id	Retrieve a specific product
POST	/api/products	Add a new product (Admin)
PUT	/api/products/:id	Update product details
DELETE	/api/products/:id	Delete a product

Cart APIs

Method	Endpoint	Description
POST	/api/cart/add	Add product to cart
GET	/api/cart	View cart items
PUT	/api/cart/update	Update product quantity
DELETE	/api/cart/remove/:id	Remove product from cart

Order APIs

Method	Endpoint	Description
POST	/api/orders	Place a new order
GET	/api/orders	View user orders
GET	/api/orders/:id	View order details
PUT	/api/orders/:id	Update order status (Admin)

Error Responses

The backend returns standard HTTP status codes for different scenarios:

- 200 OK – Request completed successfully.
- 201 Created – Resource created successfully.
- 400 Bad Request – Invalid request data.
- 401 Unauthorized – Authentication required.
- 403 Forbidden – Access denied.
- 404 Not Found – Resource not found.
- 500 Internal Server Error – Unexpected server error.

8. Authentication

Authentication and authorization in the SHOPEZ E-commerce Application are implemented using JSON Web Tokens (JWT) to ensure secure access to protected resources.

User Registration

New users can create an account by providing their name, email address, and password. Before storing the password in the database, it is encrypted using a hashing algorithm (such as bcrypt) to improve security.

User Login

During login, the user's credentials are verified against the stored data in MongoDB. If the credentials are valid, the server generates a JSON Web Token (JWT) and returns it to the client.

The frontend stores the token securely (typically in local storage or cookies) and includes it in the Authorization header for future API requests.

Example Authorization Header

```
Authorization: Bearer <JWT_TOKEN>
```

Authorization

Protected API routes verify the JWT before processing requests. Only authenticated users can access features such as:

- Viewing user profile
- Adding products to cart
- Placing orders
- Viewing order history

Administrative routes require additional role-based authorization, allowing only admin users to perform operations such as:

- Adding new products
- Updating product details
- Deleting products
- Managing users
- Managing customer orders

Security Measures

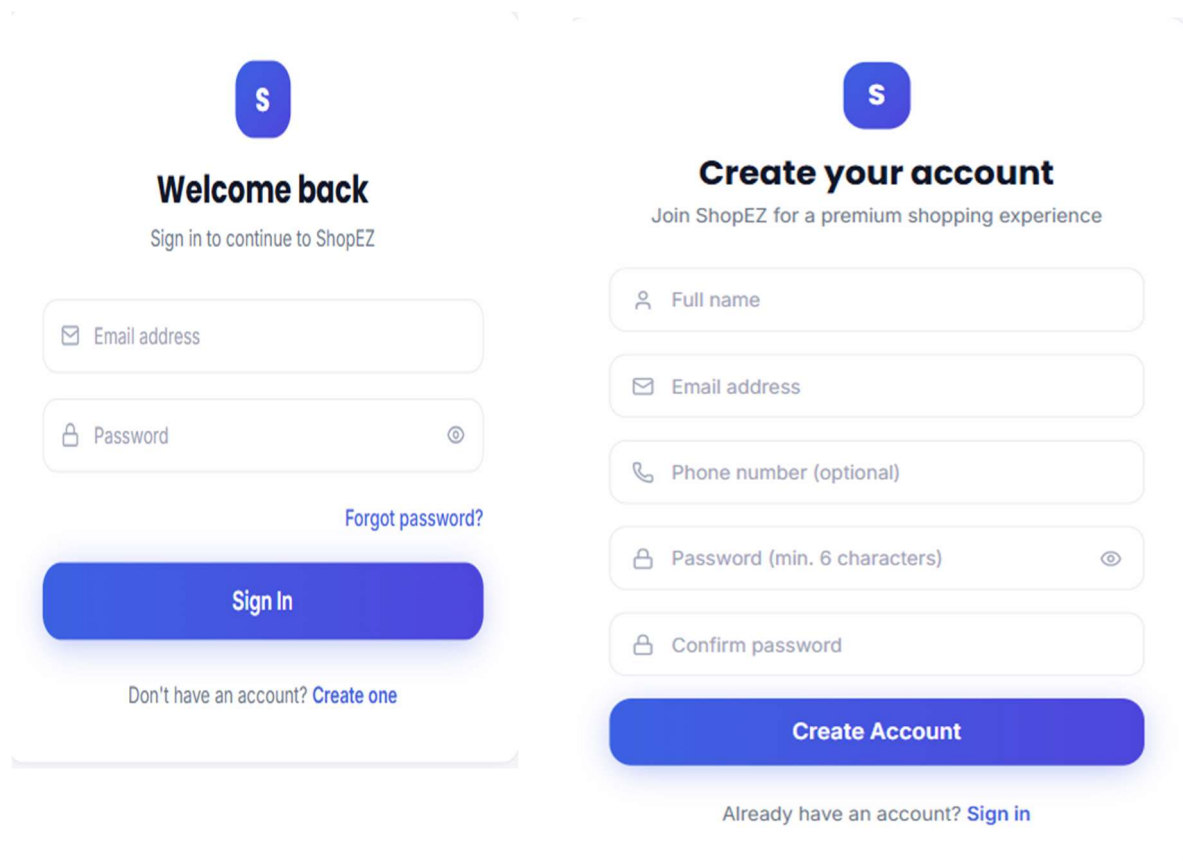
- Passwords are securely hashed before storage.
- JWT tokens are validated on every protected request.
- Unauthorized requests are rejected with appropriate HTTP status codes.
- Input validation helps prevent invalid or malicious data from entering the system.
- Middleware is used to protect sensitive API endpoints.

9. User Interface

The SHOPEZ E-commerce Application features a modern, responsive, and user-friendly interface developed using React.js. The interface is designed to provide a smooth shopping experience across desktops, tablets, and mobile devices.

Login and Registration

Users can securely create a new account or log in using their registered email address and password.



The image displays two side-by-side login and registration forms within a light gray container. Both forms feature a blue circular icon with a white 'S' at the top.

Left Form (Login):

- Title:** "Welcome back" in bold black text.
- Subtitle:** "Sign in to continue to ShopEZ" in smaller black text.
- Fields:** Two input fields. The first is labeled "Email address" with an envelope icon. The second is labeled "Password" with a lock icon and a toggle eye icon.
- Link:** A blue link "Forgot password?" is positioned below the password field.
- Button:** A large blue button labeled "Sign In" is centered below the fields.
- Footer:** A link "Don't have an account? [Create one](#)" is at the bottom.

Right Form (Registration):

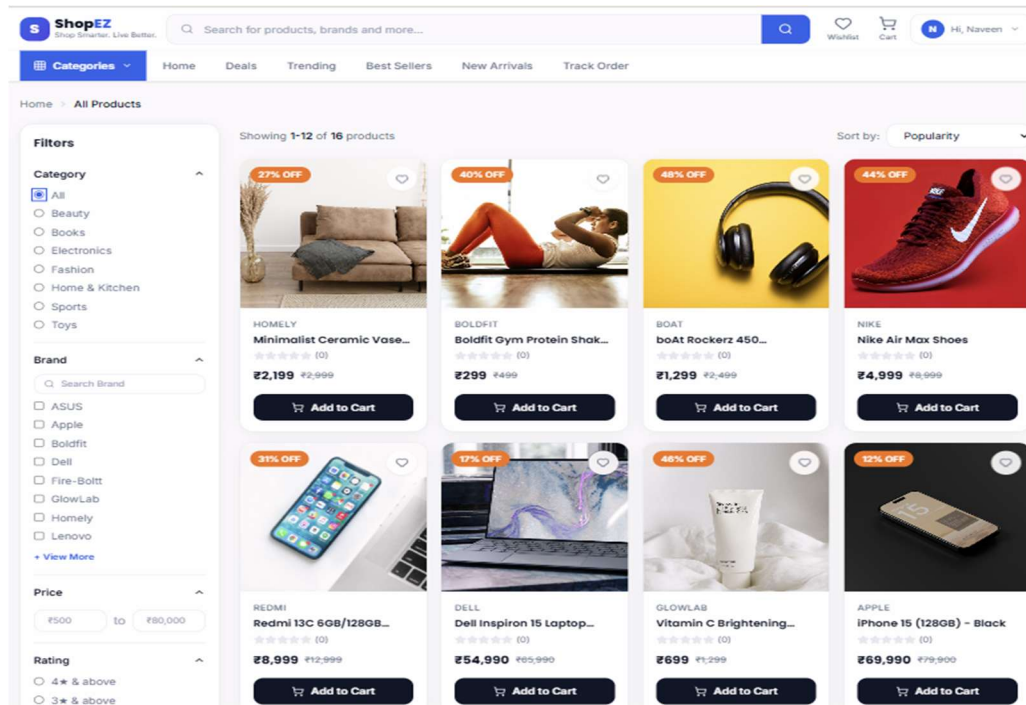
- Title:** "Create your account" in bold black text.
- Subtitle:** "Join ShopEZ for a premium shopping experience" in smaller black text.
- Fields:** Five input fields. The first is labeled "Full name" with a person icon. The second is labeled "Email address" with an envelope icon. The third is labeled "Phone number (optional)" with a phone icon. The fourth is labeled "Password (min. 6 characters)" with a lock icon and a toggle eye icon. The fifth is labeled "Confirm password" with a lock icon.
- Button:** A large blue button labeled "Create Account" is centered below the fields.
- Footer:** A link "Already have an account? [Sign in](#)" is at the bottom.

Home Page

The home page displays featured products, categories, promotional banners, and navigation options, allowing users to quickly browse available items.

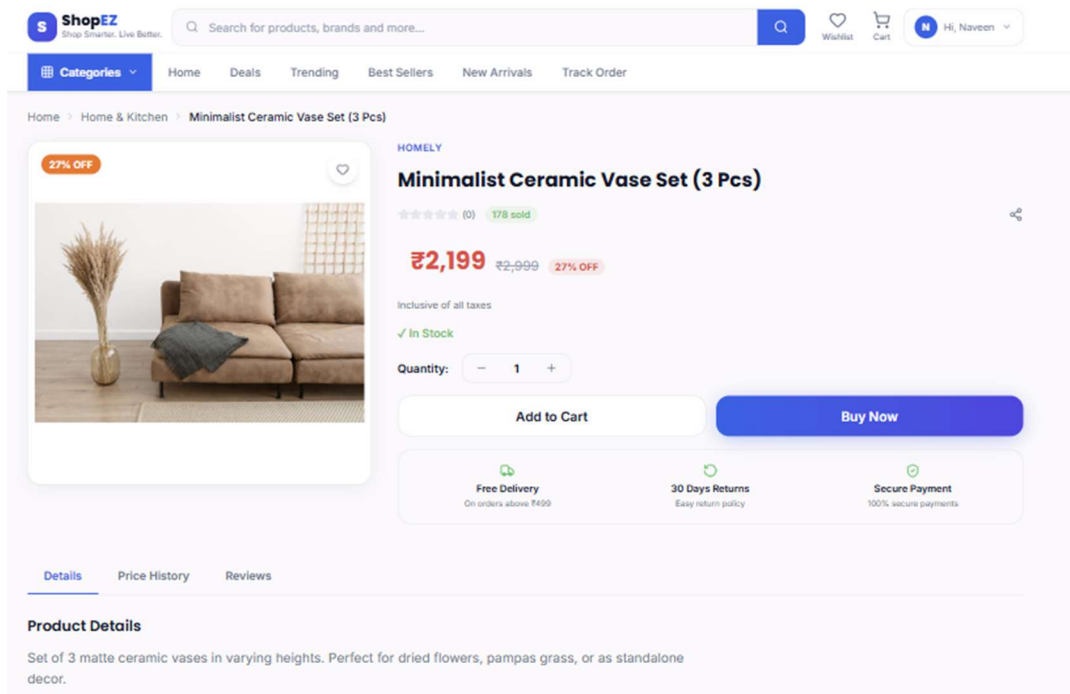
Product Listing

The product page displays available products with images, prices, descriptions, and ratings. Users can search, filter, and browse products by category.



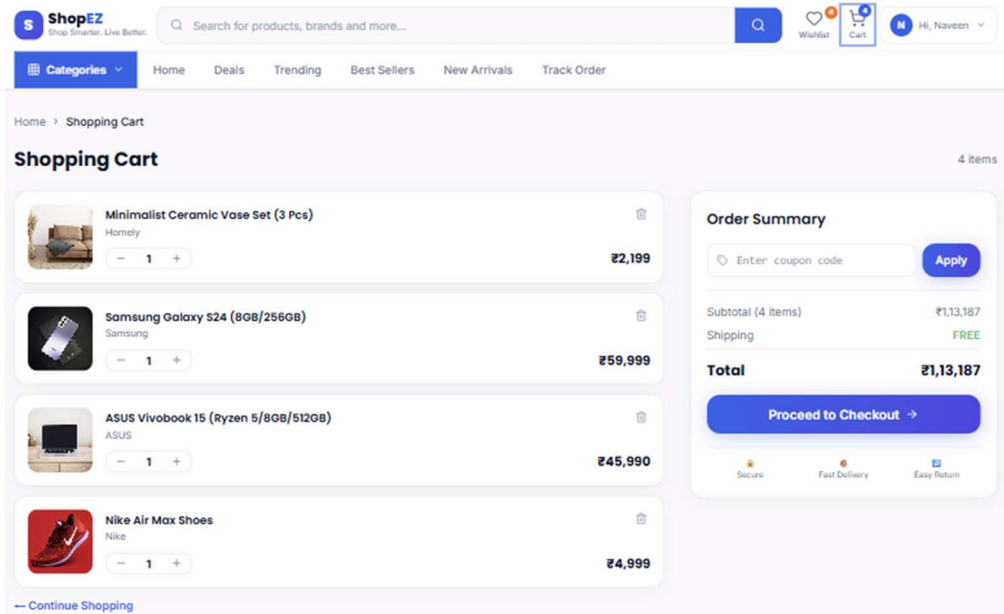
Product Details

Selecting a product opens a detailed page containing product information, images, price, availability, and an option to add the item to the shopping cart.



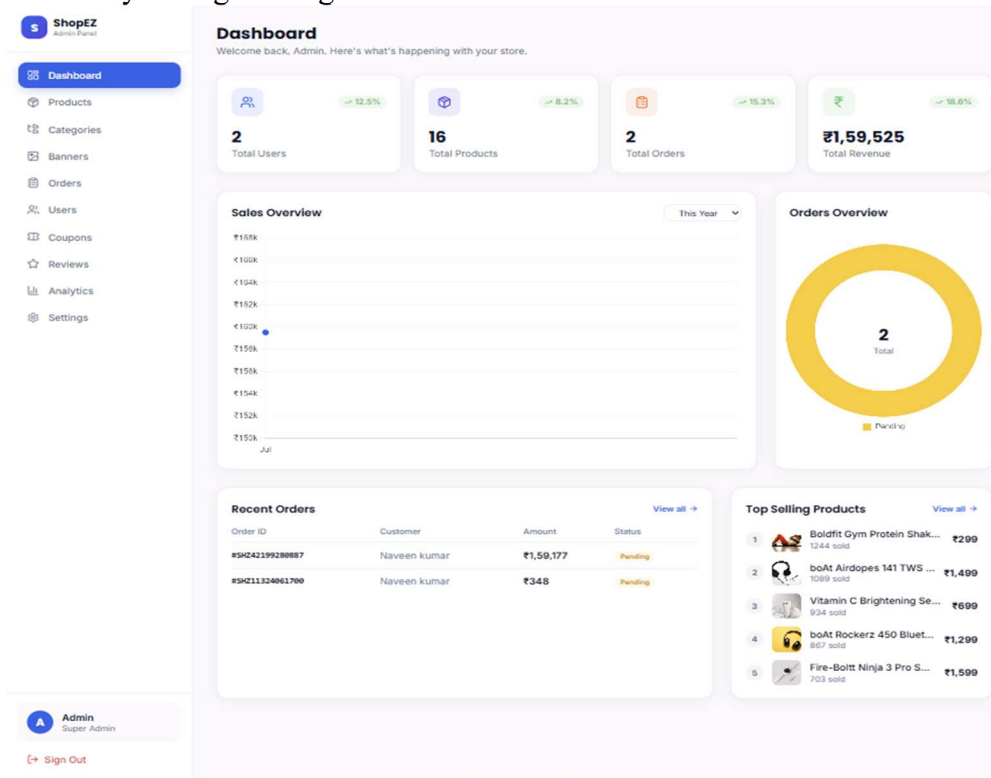
Shopping Cart

The shopping cart displays selected products, quantities, individual prices, and the total amount. Users can modify quantities or remove items before checkout.



Admin Dashboard

The admin dashboard enables administrators to manage products, users, and customer orders efficiently through an organized interface.



Checkout

The checkout page allows users to review their order, provide shipping information, and complete the purchase securely.

S

ShopEZ

Shop Smarter. Live Better.

Search for products, brands and more...

Q

Wishlist

Cart

N Hi, Naveen

Categories

Home

Deals

Trending

Best Sellers

New Arrivals

Track Order

Checkout

Address

Delivery

Payment

Review

1 Shipping Address

Full Name *

Naveen kumar

Phone *

4841689489

Address Line 1 *

123, Green Park

Address Line 2

Apartment, floor, landmark (optional)

City *

New Delhi

State *

Delhi

Postal Code *

110016

Country

India

Or choose a saved address:

Home — Naveen kumar, Chinahamsa, Palasa

Continue to Delivery

2 Delivery Options

3 Payment Method

4 Review Order

Order Summary



Minimalist Ceramic Vase Set...

Qty: 1

₹2,199



Samsung Galaxy S24 (8G...

Qty: 1

₹59,999



ASUS Vivobook 15 (Ryzen ...

Qty: 2

₹91,980



Nike Air Max Shoes

Qty: 1

₹4,999

Items (4)

₹1,59,177

Shipping

FREE

Total

₹1,59,177

Responsive Design

The application is fully responsive and adapts to different screen sizes, ensuring an optimal user experience across various devices.

10. Testing

Testing is an essential phase in the development of the SHOPEZ E-commerce Application. It ensures that all modules function correctly, the application is secure, and users experience a reliable and error-free system.

Testing Strategy

The project was tested using both manual and API testing techniques to verify the functionality of the frontend and backend components.

Functional Testing

- User Registration
- User Login and Logout
- Product Listing
- Product Search and Filtering
- Shopping Cart Operations
- Order Placement
- User Profile Management
- Admin Dashboard Operations

API Testing

REST APIs were tested using Postman to verify:

- Correct request and response formats
- Authentication using JWT
- Error handling
- CRUD operations
- HTTP status codes

User Interface Testing

The React frontend was tested to ensure:

- Responsive layouts
- Smooth navigation
- Form validation
- Proper rendering of components

- Compatibility across different browsers

Database Testing

MongoDB operations were verified for:

- Data insertion
- Data retrieval
- Data updates
- Data deletion
- Data consistency

Testing Tools

- Postman
- Google Chrome
- Mozilla Firefox
- MongoDB Compass
- Visual Studio Code Debugger

Test Results

All major modules were successfully tested. User authentication, product management, shopping cart functionality, order processing, and API communication worked as expected. Minor UI issues identified during testing were resolved before deployment.

Project Links

Demo Link:	<i>[Insert deployed application URL here]</i>
GitHub Repository:	<i>[Insert GitHub repository URL here]</i>

11. Known Issues

Although the SHOPEZ E-commerce Application is fully functional, a few limitations and minor issues have been identified during development and testing.

- Payment gateway integration has not been implemented and is currently simulated.
- Email notifications for order confirmation are not available.

- Product recommendation functionality is not included.
- Advanced search filters can be further enhanced.
- Performance may decrease when handling a very large number of concurrent users without load balancing.
- The application currently supports only basic administrator roles.
- Password recovery using email verification has not been implemented.
- Inventory synchronization for multiple sellers is not available.

These issues do not affect the core functionality of the application but can be addressed in future versions.

12. Future Enhancements

The SHOPEZ E-commerce Application can be further improved by adding advanced features and optimizing performance. Some of the proposed future enhancements include:

- Integrate secure online payment gateways such as Stripe, Razorpay, or PayPal.
- Implement email verification and password recovery functionality.
- Add product reviews and customer ratings.
- Develop a wishlist feature for saving favorite products.
- Introduce AI-based product recommendations based on user preferences.
- Enhance product search with advanced filtering and sorting options.
- Implement real-time order tracking using WebSockets.
- Add push notifications for order updates and promotional offers.
- Develop a multi-vendor marketplace with seller registration and management.
- Improve application performance using caching and database optimization.
- Deploy the application using cloud platforms such as AWS, Azure, or Render.
- Integrate analytics dashboards for administrators to monitor sales and customer activity.
- Add multilingual support and multiple currency options to serve a broader audience.
- Strengthen security with two-factor authentication (2FA) and enhanced access control.

Implementing these enhancements will improve the scalability, usability, security, and overall user experience of the SHOPEZ E-commerce Application, making it more suitable for real-world commercial deployment.